

NR2DT-07: Step 7 — Migrate Logs, Tags & Drop Rules

Series: NR2DT — New Relic to Dynatrace Migration Steps | **Notebook:** 7 of 10 |
Created: April 2026 | **Last Updated:** 10/05/2026

Overview

Goal of this step: complete Wave 5 — reconfigure log forwarders to ship to Dynatrace, apply OpenPipeline parsing/drop/enrichment rules, and confirm cost-optimization patterns are working.

Procedural — see **NRLC-07** (Logs, Tags & Drop Rules) for component depth.

Table of Contents

- [1. Apply OpenPipeline Configuration](#)
 - [2. Reconfigure Log Forwarders](#)
 - [3. Validate Volumes and Drops](#)
 - [4. Verify Tag Enrichment](#)
 - [5. Step Exit Criteria](#)
-

Prerequisites

Requirement	Details
Audience	Migration lead + assigned engineer for this step

Requirement	Details
Completed	NR2DT-06 — Migrate Synthetics, SLOs, Workloads
Format	Procedural step — use as a runbook; defer to NRLC for depth
NRLC deep dives	NRLC-07 (Logs, Tags & Drops)

1. Apply OpenPipeline Configuration

```
# The component names this tool version accepts
python3 migrate.py migrate --list-components
```

The log components are named `log_parsing`, `drop_rules` and `tags`. `logs`, `drops` and `parsing` are not component names, and the tool skips an unknown name without an error — so the command this step used to show applied nothing.

At tool v3.0.0 (commit `78cbfce`) the `migrate` command does not migrate the log components either. Its export and transform phases have branches only for dashboards, alerts, synthetics, SLOs, workloads and notification channels, so `--components log_parsing,drop_rules,tags` completes without exporting, transforming or importing a single log rule. (The tool's README lists log rules as fully supported; the CLI path at this commit does not reach those transformers.) Re-check `--list-components` and the changelog in your checkout. Until it changes, build the OpenPipeline configuration from the NR2DT-03 §5 design yourself, with Terraform or the OpenPipeline UI.

Sources: [migrate.py @ 78cbfce \(tool repo, GitHub\)](#), [config/settings.py @ 78cbfce \(tool repo, GitHub\)](#), read 10/05/2026.

This step applies:

- Drop rules (NRQL filters → OpenPipeline `filterout` processors)
- Parsing rules (Grok → DPL parsers)
- Tag rules (NR entity tags → OpenPipeline enrichment rules emitting attributes on ingested data)

2. Reconfigure Log Forwarders

Reconfigure each log shipper to point at DT instead of (or in addition to) NR:

Source	Reconfigure
Filebeat / Fluent Bit	DT log ingest endpoint or OneAgent
Lambda log forwarder	DT Lambda extension or CloudWatch → Firehose → OpenPipeline
K8s log integration	DynaKube + log monitoring
Syslog	Environment ActiveGate syslog receiver (UDP 514 / TCP 601 by default) → OpenPipeline processing
HTTP/JSON	DT Generic Log Ingest API

Syslog is not received by OpenPipeline directly: *"Syslog ingestion is performed by an ActiveGate"*, and *"Multi-environment ActiveGates do not support syslog ingestion"* — plan an Environment ActiveGate with syslog enabled for each receiving environment.

Sources: [Syslog ingestion with ActiveGate \(DT docs\)](#), read 10/05/2026.

Run dual-shipping (both NR and DT receive) for 1–2 weeks. NR is silenced after volume validation.

3. Validate Volumes and Drops

Volume parity

```
fetch logs, from:-1d
| summarize records = count(), by:{dt.system.bucket}
| sort records desc
```

Compare against NR's `SELECT count(*) FROM Log SINCE 1 day ago`. Volumes should match within $\pm 5\%$.

Drop rule effectiveness

Confirm dropped patterns are being filtered. Use the **same matcher as the drop rule** (NR2DT-03 S5 drops `contains(content, "GET /health")`):

```
// Same matcher as the drop rule: n should be 0 once the rule is active
fetch logs, from:-1h
| filter contains(content, "GET /health")
| summarize n = count()
```

`n` should be 0 once the drop rule is working. A broader `contains(content, "health")` also counts lines the rule never drops — on the validation tenant (10/05/2026) it matched 1,891 lines in an hour while `"GET /health"` matched none — so it reads as a failure even where the rule has nothing left to drop.

Cost-optimization spot-check

```
fetch logs, from:-24h
| summarize total = count(),
    droppable = countIf(loglevel == "DEBUG" OR contains(content, "GET
/health"))
| fieldsAdd dropPercent = round((toDouble(droppable) / toDouble(total)) * 100,
decimals: 2)
```

Confirms how much volume is left to drop (target: 0% droppable after Wave 5).

4. Verify Tag Enrichment

Tags become OpenPipeline enrichment attributes attached to ingested data. Verify:

```
fetch logs, from:-1h
| filter environment == "prod"
| summarize count(), by:{environment}
```

Should return enriched data with the expected environment value.

Workload identifiers should similarly be queryable:

```
fetch logs, from:-1h
| filter workload.name == "checkout"
| summarize count()
```

5. Step Exit Criteria

S7 — Logs / Tags / Drops Migrated

- [] All log forwarders reconfigured to DT
- [] OpenPipeline parsing, drop, and enrichment rules applied
- [] W5 volume parity validated ($\pm 5\%$ vs NR)
- [] Drop rules confirmed working (target patterns absent)
- [] Tag enrichment producing expected attributes on ingested data

Next step: NR2DT-08 — Validate.

This notebook was AI-generated from community-submitted and publicly available sources, including the open-source [NewRelic-to-Dynatrace-Migration-Utilities](#), [nrql-engine](#), and [nrql-translator](#) projects. This notebook series is not officially supported by Dynatrace or New Relic. Always verify information against the official [Dynatrace documentation](#) and [New Relic documentation](#).